

Chương 2. Kiểu con trỏ

2.1 Khai báo :

T* P;

- Giá trị chứa trong P là *địa chỉ* của vùng nhớ chứa giá trị có kiểu T,
- Ta cũng nói P là con trỏ trỏ tới vùng nhớ chứa giá trị có kiểu T, hay **P là con trỏ có kiểu T**.
- Giá trị NULL : Khi P có giá trị này thì xem như P không trỏ vào đâu cả. Câu lệnh thường dùng

if (P!=NULL) { ... }

VD :

```
int *P; // là con trỏ có kiểu int
```

```
int x;
```

```
x = 10;
```

```
P = &x ; // P chứa địa chỉ của biến x. Vùng nhớ được trỏ bởi P là x.
```

```
// Thường được biểu diễn P •  $\longrightarrow$  x
```

2.2 Truy xuất vùng nhớ trỏ bởi con trỏ:

Cho khai báo :

T* P ;

vùng nhớ được trỏ bởi P là *P.

VD :

```
int* P; // là con trỏ có kiểu int
```

```
int x;
```

```
x = 10;
```

```
P = &x ; // P chứa địa chỉ của biến x.
```

```
printf(“%d”, *P); // Viết ra màn hình giá trị của vùng nhớ được trỏ bởi P,  
tức là x. Trên màn hình là 10.
```

VD :

```
int* P; // là con trỏ có kiểu int
```

```
int x;
```

```
x = 10;
```

```
P = &x ; // P chứa địa chỉ của biến x.
```

```
*P = *P + 2; // Thay đổi giá trị của vùng nhớ được trỏ bởi P, tức là x.
```

```
printf("%d", x); // Trên màn hình là 12
```

Lệnh **scanf**("—", &x) : nhập giá trị vào vùng nhớ có địa chỉ là địa chỉ của x, tức là nhập giá trị cho biến x.

VD : Đoạn chương trình sau :

```
int x;  
scanf("%d", &x);  
printf("%d", x);
```

tương đương với

```
int x, *P;  
P=&x;  
scanf("%d", P);  
printf("%d", x);
```

2.3 Mảng và con trỏ:

Cho khai báo mảng :

T M[n] ;

- Ta có một mảng n phần tử kiểu T. **M** là con trỏ trỏ tới phần tử đầu tiên của **mảng** (chứa địa chỉ của phần tử đầu tiên của mảng).
- $(M + i)$, $i = 0, \dots, n-1$, là địa chỉ của phần tử thứ i của mảng. Vậy $*(M+i)$ là $M[i]$.
- Mảng được khai báo như trên còn được gọi là *mảng tĩnh*.

VD 1:

```
int M[3];
```

```
*(M+0)=10; //  $\Leftrightarrow *M=10$  ;  $\Leftrightarrow M[0]=10$  ;
```

```
*(M+1)=20; //  $\Leftrightarrow M[1]=20$  ;
```

```
*(M+2)=30; //  $\Leftrightarrow M[2]=20$  ;
```

VD 2:

```
int M[3], *P;
```

```
P=M;
```

```
*(P+0)=10; //  $\Leftrightarrow P[0]=10$ ;  $\Leftrightarrow M[0]=10$  ;
```

```
*(P+1)=20; //  $\Leftrightarrow P[1]=20$ ;  $\Leftrightarrow M[1]=20$  ;
```

```
*(P+2)=30; //  $\Leftrightarrow P[2]=20$ ;  $\Leftrightarrow M[2]=20$  ;
```

2.4 Cấp phát vùng nhớ cho con trỏ :

```
#include <malloc.h>
```

```
int main(. . .)
```

```
{
```

```
    T *P;
```

```
    P=(T*)malloc(n*sizeof(T)); (1)
```

```
    . . .
```

```
}
```

- (1) sẽ cấp phát n ô nhớ có kiểu là T và con trỏ P sẽ trỏ tới ô nhớ đầu tiên,
- P là mảng kiểu T, được gọi là *mảng động*, các phần tử của mảng là P[i], i=0, . . ., n-1,
- Nếu không cấp phát được thì P có giá trị NULL.

Ví dụ :

```
#include <malloc.h>
```

```
int main(int argc, char* argv[])
```

```
{
```

```
    int *P;
```

```
    P=(int*)malloc(1*sizeof(int)); //  $\Leftrightarrow$  P=(int*)malloc(sizeof(int));
```

```
    *P=10;
```

```
    printf("*P= %d\n",*P);
```

```
    // Trên màn hình : 10
```

```
    return 0;
```

```
}
```


Ví dụ :

```
#include <malloc.h>
```

```
int main(int argc, char* argv[])
```

```
{
```

```
    int *P;
```

```
    P=(int*)malloc(sizeof(int));
```

```
    P[0]=10;
```

```
    printf("*P= %d\n",*P);
```

```
// Trên màn hình : 10
```

```
    return 0;
```

```
}
```

Ví dụ :

```
#include <malloc.h>
```

```
int main(int argc, char* argv[])
```

```
{
```

```
    int *P;
```

```
    P=(int*)malloc(3*sizeof(int));
```

```
    P[0]=10; P[1]=100; P[2]=1000;
```

```
    printf("%d, %d, %d\n",P[0], P[1], P[2]); // Trên màn hình : 10, 100, 1000
```

```
    return 0;
```

```
}
```

Ví dụ :

```
#include <malloc.h>
```

```
int main(int argc, char* argv[])
```

```
{
```

```
    int *P;
```

```
    P=(int*)malloc(3*sizeof(int));
```

```
    P[0]=10; P[1]=100; P[2]=1000;
```

```
    printf("%d, %d, %d\n",*P, *(P+1), *(P+2)); // Trên màn hình : 10, 100, 1000
```

```
    return 0;
```

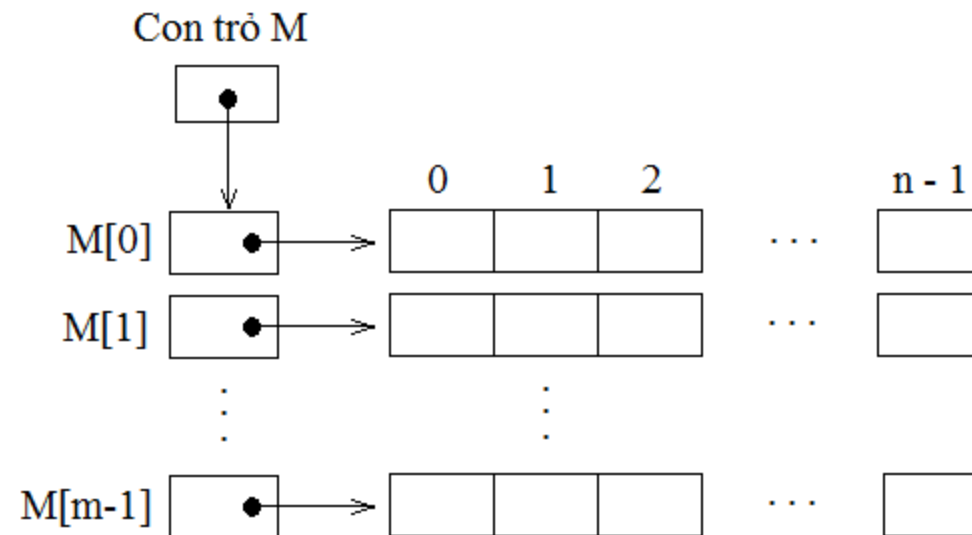
```
}
```

2.5 Con trỏ và mảng 2 chiều :

Cho khai báo :

T M[m][n];

M là con trỏ có kiểu con trỏ (có kiểu T).

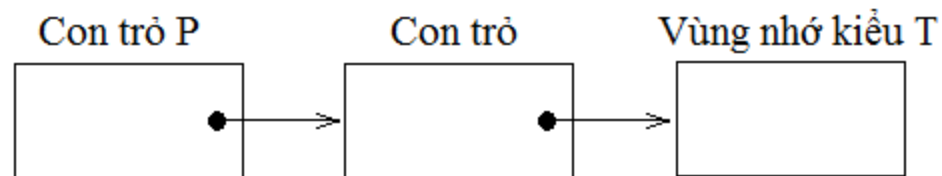


2.6 con trỏ có kiểu con trỏ (kiểu T) :

2.6.1 Khai báo con trỏ có kiểu con trỏ :

T **P;

- P là con trỏ (chứa địa chỉ) trỏ tới vùng nhớ, (vùng nhớ này) là con trỏ trỏ tới vùng nhớ chứa giá trị có kiểu T.



- *P là vùng nhớ được trỏ bởi P, gọi vùng nhớ này là Q, thì **P là *Q, là vùng nhớ kiểu T.

Ví dụ :

```
int x , *Q, **P;  
x = 5;  
Q = &x;  
P = &Q;  
printf(“%d\n”, **P);
```

Ví dụ :

```
int x , *Q, **P;  
x = 5;  
Q = &x;  
P = &Q;  
**P = **P +1;  
printf(“%d\n”, x);
```

2.6.2 Cấp phát vùng nhớ cho *Con trỏ có kiểu con trỏ* (kiểu T) :

Cho khai báo :

T **P;

$P = (T^{**})\text{malloc}(\mathbf{m} * \text{sizeof}(T^{*}));$ (1)

$P[i] = (T^{*})\text{malloc}(\mathbf{n}_i * \text{sizeof}(T));$ // $i = 0, \dots, m-1$ (2)

- (1) : P là con trỏ, trỏ tới phần tử đầu tiên của mảng m phần tử. Mỗi phần tử trong mảng có kiểu *con trỏ kiểu T*, P[i],
- (2) : Cấp phát mảng n_i phần tử kiểu T cho P[i] (P[i] trỏ tới phần tử đầu tiên của mảng).

```
T **P;
```

```
P=(T**)malloc(m*sizeof(T*)); (1)
```

```
P[i] = (T*)malloc(ni*sizeof(T)); // i = 0, . . ., m-1 (2)
```

Ví dụ

```
int **P;
```

```
P=(int**)malloc(2*sizeof(int*)); (1)
```

```
P[0] = (int*)malloc(2*sizeof(int)); //  $\Leftrightarrow$  P[0][0], P[0][1] (2)
```

```
P[1] = (int*)malloc(3*sizeof(int)); //  $\Leftrightarrow$  P[1][0], P[1][1], P[1][2] (2)
```

```
*(P[1]+1)=10;
```

```
printf("kq = %d\n",P[1][1]);
```


2.7 Mảng các con trỏ kiểu T :

Cho khai báo :

T *M[n];

- M[i], i=0, . . ., n-1, là con trỏ kiểu T,
- M[i] = (T*)malloc(n_i*sizeof(T)); // i = 0, . . ., n-1

VD :

```
int *M[2];
```

```
M[0] = (int*)malloc(2*sizeof(int)); // ⇔ M[0][0], M[0][1]
```

```
M[1] = (int*)malloc(3*sizeof(int)); // ⇔ M[1][0], M[1][1], M[1][2]
```

```
*(M[1]+1)=10;
```

```
printf("kq = %d\n",M[1][1]);
```

2.8 Giải phóng vùng nhớ cấp phát cho p :

T *p;

p=(T*)malloc(N*sizeof(T));

...

free(p);

2.9 Con trỏ là tham số của chương trình con :

Trường hợp 1 :

```
void P(int *ptr)
{
    ptr[0]=12;
    ptr[1]=34;
}
```

```
int main(int argc, char* argv[])
{
    int M[2] = {1, 2};

    P(M);

    printf("M[0]=%d", M[0]);
    printf("M[1]=%d", M[1]);
}
```

2.9 Con trỏ là tham số của chương trình con :

Trường hợp 2 :

```
void P(int *ptr)
{
    ptr=(int*)malloc(2*sizeof(int));
    ptr[0]=12;
    ptr[1]=34;
}
```

```
int main(int argc, char* argv[])
{
    int M[2] = {1,2};

    P(M);

    printf("M[0]=%d",M[0]); // 1
    printf("M[1]=%d",M[1]); // 2
}
```

2.9 Con trỏ là tham số của chương trình con :

Trường hợp 3 : Cấp phát vùng nhớ cho M bằng chương trình con. M là con trỏ.

```
void P(int *ptr)
{
    ptr=(int*)malloc(2*sizeof(int));
    ptr[0]=12;
    ptr[1]=34;
}
```

```
int main(int argc, char* argv[])
{
    int *M;

    P(M); // Error

    printf("M[0]=%d",M[0]); // 1
    printf("M[1]=%d",M[1]); // 2
}
```

2.9 Con trỏ là tham số của chương trình con :

Trường hợp 4 : Cấp phát vùng nhớ cho M bằng chương trình con. M là con trỏ.

```
void P(int **ptr)
{
    *ptr=(int*)malloc(2*sizeof(int));
    (*ptr)[0]=12;
    (*ptr)[1]=34;
}
```

```
int main(int argc, char* argv[])
{
    int *M;

    P(&M);

    printf("M[0]=%d",M[0]); // 12
    printf("M[1]=%d",M[1]); // 34
}
```

2.9 Con trỏ là tham số của chương trình con :

Trường hợp 5 : Dùng C++ cấp phát vùng nhớ cho M bằng chương trình con. M là con trỏ.

```
void P(int* &ptr)
{
    ptr=(int*)malloc(2*sizeof(int));
    ptr[0]=12;
    ptr[1]=34;
}
```

```
int main(int argc, char* argv[])
{
    int *M;

    P(M);

    printf("M[0]=%d",M[0]); // 12
    printf("M[1]=%d",M[1]); // 34
}
```

2.9 Con trỏ là tham số của chương trình con :

Trường hợp 5 : Dùng C++ cấp phát vùng nhớ cho M bằng chương trình con. M là con trỏ.

```
typedef int* INT_PTR;
```

```
void P(INT_PTR &ptr)  
{  
    ptr=(int*)malloc(2*sizeof(int));  
    ptr[0]=12;  
    ptr[1]=34;  
}
```

```
int main(int argc, char* argv[])  
{  
    int *M; // INT_PTR M;  
  
    P(M);  
  
    printf("M[0]=%d",M[0]); // 12  
    printf("M[1]=%d",M[1]); // 34  
}
```


2.9 Con trỏ là tham số của chương trình con :

Trường hợp 6 : Dùng C++ cấp phát vùng nhớ cho M bằng chương trình con. M là con trỏ.

```
void P(int **&ptr)
{
    ptr=(int**)malloc(2*sizeof(int*));
    ptr[0]=(int*)malloc(3*sizeof(int));
    ptr[1]=(int*)malloc(3*sizeof(int));
}
```

```
int main(int argc, char* argv[])
{
    int **M;

    P(M);
    ...
}
```

Ví dụ :

```
void sinh_tohop(int n, int m, int **&S)
{  int i, j, C;
   C= n!/(m!(n-m!)) ; //  $C = C_m^n$ 
   // Cấp phát mảng 2 chiều, kích thước C x m
   S=(int**)malloc(C*sizeof(int*));
   for(i=0; i < C; i++) S[i]=(int*)malloc(m*sizeof(int));
   B2. for(i=0; i < m; i++) S[0][i]=i;
   B3. for(i=1; i < C; i++)
   {
      B4. copy(S[i-1], S[i], m);
      B5. for(j=m-1; j>=0; j--) if (S[i][j]!=(n-1)-(m-1)+j) break;
      B6. S[i][j]++;
      B7. for(int r=j+1;r<m; r++) S[i][r]=S[i][r-1]+1;
   }
}
```

```
void main()
{
   int **S;
   sinh_tohop(5, 3, S);
   ...
   free(S);
}
```

2.9 Con trỏ là tham số của chương trình con :

Trường hợp 6 : Dùng C++ cấp phát vùng nhớ cho M bằng chương trình con. M là con trỏ.

```
typedef int** INT_PTR;
```

```
void P(INT_PTR &ptr)  
{  
    ptr=(int**)malloc(2*sizeof(int*));  
    ptr[0]=(int*)malloc(3*sizeof(int));  
    ptr[1]=(int*)malloc(3*sizeof(int));  
}
```

```
int main(int argc, char* argv[])  
{  
    int **M; // INT_PTR M;  
  
    P(M);  
    ...  
}
```

Chuỗi và con trỏ (Tham khảo):

VD :

```
char S[30], *P;  
strcpy(S, "ABCDEFGH");  
P = (S+2);  
printf("%s\n", P); // Trên màn hình là : CDEFGH
```

VD :

```
char S[30], *P;  
strcpy(S, "ABCDEFGH");  
P = (S+2);  
strcpy(P, "1234");  
printf("%s\n", S); // Trên màn hình là : ?
```